Learned Index Structures: Recursive Model Index vs. B-Tree

CS631 Course Project, Checkpoint 1 progress report
Paper: Kraska, Beutel, Chi, Dean, Polyzotis, *The Case for Learned Index Structures*, SIGMOD 2018
Team: _____ (23b1024) _____ (23b0939)

1. Goal

The paper argues that an index is a model mapping a key to its position in sorted data, i.e. it approximates the CDF of the keys. We are reproducing its range-index results (Section 3): a 2-stage **Recursive Model Index (RMI)** that predicts a key's position, compared against a read-optimized **B-Tree** on lookup time and index size.

2. Implementation so far (C++17, about 2,000 lines)

RMI (`src/rmi.hpp, rmi.cpp`; paper §3.2, §3.4)

- **Stage 1:** one linear model maps a key to one of K stage-2 models. It is fit on either key or log(key), whichever spreads keys more evenly across the K models.
- **Stage 2:** K linear models, each a closed-form least-squares fit on its own keys (centred sums in `long double`, since keys reach 2^{63}).
- **Error bounds:** each model stores its worst over- and under-prediction. A lookup binary-searches only `[pred + min_err, pred + max_err]`, so every present key is guaranteed to be found.
- Stage 1 is monotone, so each stage-2 model owns a contiguous slice of the array. Training needs no per-model copies, which lets us scale to 200M keys.

B-Tree baseline (`src/btree.hpp, btree.cpp`): read-only, built bottom-up with 100% full pages of 32/64/128/256 keys. The leaf level is the sorted array itself, and each upper level is one cache-aligned key array. Child positions are computed, so no pointers are stored. This makes the baseline both smaller and faster than a textbook B-Tree.

Benchmark (`bench/benchmark.cpp`): both indexes are in one binary with the same flags and the same branch-free binary search. Every key is checked to be found at its exact position, and 1,000 absent keys are checked to return "not found", before any timing. Timing is the median of 5 runs of 1M shuffled random lookups. A second standalone RMI (`rmi/`) reports error statistics up to 200M keys and supports exponential search.

Datasets: *dense* (0..N−1), *uniform_sparse* (random 64-bit keys), *lognormal* (heavy-tailed, as in the paper). N = 10M and 100M (200M for RMI statistics), K = 1k, 10k, 100k. Machine: Intel i7-13620H, g++ 13.3, `-O3 -march=native`.

3. Results

Correctness: all 69 configurations (42 RMI/B-Tree comparisons plus 27 RMI-only runs) found 100% of keys at their exact positions and returned no false positives.

Dataset (N = 100M)	B-Tree, 128-key pages	Best RMI	Speedup	Index size
dense	367 ns, 6.0 MB	K=1k: 33 ns, 23 KB	11.1×	262× smaller
uniform_sparse	373 ns, 6.0 MB	K=100k: 111 ns, 2.3 MB	3.4×	2.6× smaller
lognormal	377 ns, 6.0 MB	K=100k: 196 ns, 2.3 MB	1.9×	2.6× smaller

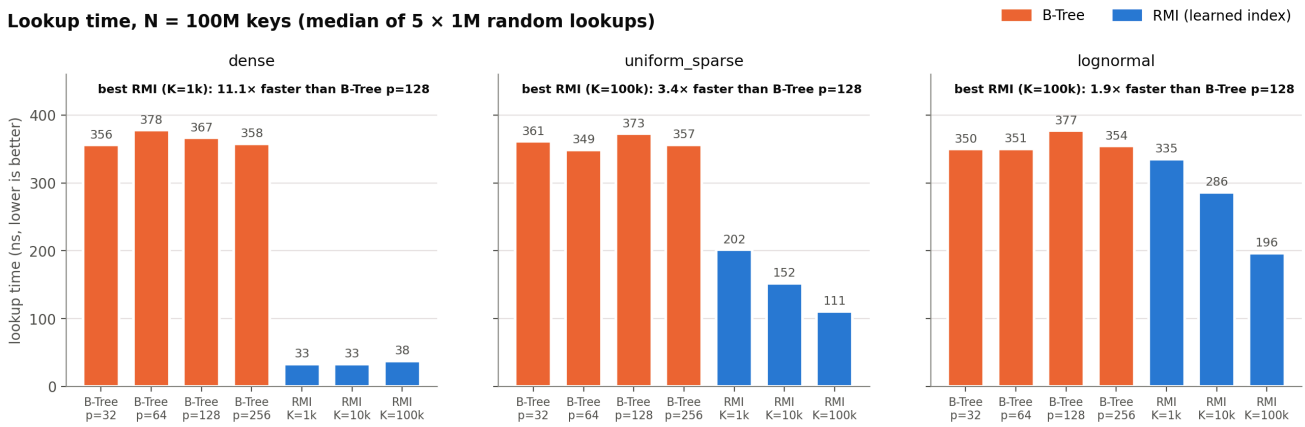


Figure 1. Lookup time at N = 100M keys (B-Tree in orange, RMI in blue). Every RMI configuration beats every B-Tree page size, at both 10M and 100M keys.

Observations

- **Model execution is cheap.** The two RMI models take 4–15 ns, only 2–14% of a lookup. The rest is the search, which waits on main memory. The B-Tree spends about 80 ns just walking from root to leaf.
- **A perfect model still pays one memory access.** On dense data the prediction error is 0, yet 29 of the 33 ns is fetching the key from RAM.
- **Lognormal is the weak spot.** More models shrink the mean error window (7,425 to 33 positions at 100M), but the worst single model stays at about 1.46M positions at 200M for every K. A linear stage 1 cannot follow the lognormal S-shaped CDF.
- **The size advantage depends on N.** RMI size is fixed by K while the B-Tree grows with N. At 10M keys, K=100k is 3.8× *larger* than the B-Tree, so the fastest RMI is not always the smallest.

4. Remaining work (next checkpoint)

Feature	Paper	Purpose
Small neural network (or multivariate) stage 1	§3.2	Learn the lognormal curve; fix the worst models
Hybrid index (Algorithm 1)	§3.3	Replace stage-2 models with error above a threshold by B-Trees
Model-biased binary and biased quaternary search	§3.4	Start the search at the predicted position
Real dataset (SOSD: books / osm / fb)	§3.7.1	The paper's Weblogs and Maps datasets are not public
K = 10k, 50k, 100k, 200k	§3.7.1	Match the paper's stage-2 sizes

Stretch goals, if time allows: learned hash maps (§4) and learned Bloom filters (§5).